

CO3 - ASSESSMENT TOOL 3

Exception Handling in Thread Pool Worker Threads: execute() vs submit()/Future

Name	M. Yogeshwaran
Reg No	192524363
Course Name	Java Programming
Course Code	CSA0924

1. What Happens When an Exception Is Thrown Inside run()

Every thread in a thread pool (an `ExecutorService` backed by a `ThreadPoolExecutor`) is a worker thread that repeatedly pulls `Runnable/Callable` tasks off a shared work queue and invokes their `run()` (or `call()`) method. What happens when that code throws an exception depends on the type of exception, and on how the task was submitted to the pool.

1.1 Unchecked Exceptions (`RuntimeException` / `Error`) in a `Runnable`

A `Runnable`'s `run()` method cannot declare checked exceptions, so only unchecked exceptions (`RuntimeException`) or `Errors` can propagate out of it. When such an exception escapes `run()` uninterrupted:

- The worker thread's run loop, inside `ThreadPoolExecutor.runWorker()`, catches the throwable, calls `afterExecute(task, throwable)`, and then lets the thread terminate.
- The pool detects the dying worker thread and replaces it with a brand-new worker thread, so the pool's configured size is maintained.
- By default the exception is printed to `System.err` via the thread's uncaught exception handler (`Thread.UncaughtExceptionHandler`) — it is NOT silently swallowed, but it is also not delivered back to the code that submitted the task, because that code already returned from `execute()` long before the task ran.
- Other queued/running tasks in the pool are unaffected; only the one worker thread that hit the exception is torn down and replaced.

1.2 Key Point — the Pool Survives, the Thread Does Not

A `ThreadPoolExecutor` never lets a single misbehaving task crash the whole pool. It isolates each task's execution inside the worker's run loop with a `try/catch`, so one thrown exception only costs the pool one worker thread, which is immediately replenished.

2. `execute()` vs `submit()`

Both methods hand a task to the pool for asynchronous execution, but they differ in return type, exception signature, and — most importantly — how exceptions become visible to the caller.

Aspect	<code>execute(Runnable)</code>	<code>submit(Runnable/Callable)</code>
Declared in	<code>Executor</code> interface	<code>ExecutorService</code> interface
Return value	<code>void</code> — nothing returned	<code>Future<?></code> or <code>Future<V></code> — a handle to the pending result
Accepts	<code>Runnable</code> only	<code>Runnable</code> , <code>Runnable+result</code> , or <code>Callable</code>
Exception propagation	Not returned to caller. Handled by the worker's <code>afterExecute()/uncaught-exception handler</code> ; caller has no direct way to know a task failed.	Captured, stored inside the <code>Future</code> , and only thrown to the caller when <code>Future.get()</code> is invoked (wrapped in <code>ExecutionException</code>).
Fire-and-forget?	Yes — natural fit when you don't need a result or failure notification	No — designed for cases where you want to check completion, cancel, or retrieve a result/error

3. The Role of Future

When a task is given to `submit()`, the executor wraps it in a `FutureTask` before queuing it. `FutureTask`'s `run()` method itself contains a `try/catch`: it invokes the underlying `Callable/Runnable`, and if it throws, `FutureTask` catches the throwable and stores it as the task's outcome instead of letting it escape to the worker thread.

- Because `FutureTask` swallows the exception internally, the worker thread's run loop sees a normal return — the worker thread is NOT killed and does not need to be replaced.
- The stored exception stays dormant until the caller calls `future.get()`, at which point it is rethrown wrapped in an `ExecutionException`, with the original exception available via `getCause()`.
- `future.get(timeout, unit)` additionally can throw `TimeoutException` if the task hasn't finished; `get()` also throws `InterruptedException` if the waiting thread is interrupted, and `CancellationException` if the task was cancelled.
- If `get()` is never called, an exception thrown inside the task is effectively lost — no stack trace is printed anywhere, which is a common source of silently failing `submit()`-based code.

3.1 Example

```
ExecutorService pool = Executors.newFixedThreadPool(4);

// execute(): fire-and-forget, exception goes to the
// default uncaught exception handler, not to this code
pool.execute(() -> { throw new RuntimeException("boom-execute"); });

// submit(): exception is captured inside the Future
Future<?> future = pool.submit(() -> { throw new RuntimeException("boom-
submit"); });
try {
    future.get();           // rethrown here
} catch (ExecutionException e) {
    System.out.println("Caught: " + e.getCause());
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

4. Summary

- `execute()`: exception surfaces on the worker thread itself, kills and replaces that thread, and is only visible via the thread's uncaught-exception handler (e.g. console stack trace) — the calling code never sees it directly.
- `submit()`: exception is caught by `FutureTask`, stored inside the `Future`, and the worker thread survives; the caller must call `Future.get()` to observe the failure (as an `ExecutionException`).
- Pool integrity: regardless of which method is used, the `ThreadPoolExecutor` as a whole keeps running; at most one worker thread is affected, and it is transparently replaced when using `execute()`.
- Best practice: use `submit()` with `Future` (or `CompletableFuture`) when task failures must be detected or logged; always call `get()` or otherwise inspect the `Future`, and consider a custom `Thread.UncaughtExceptionHandler` or a wrapped `Runnable` when using `execute()` to avoid silent failures.

5. Checked Exceptions, Callable, and invokeAll() / invokeAny()

Unlike Runnable, the Callable<V> interface's call() method is declared to throw the checked exception Exception. This matters because it changes what a pool can do with the failure and how bulk-submission methods report it.

5.1 Callable vs Runnable Exception Signatures

- Runnable.run(): throws nothing checked — only unchecked RuntimeException/Error can escape, which is why execute() has no compile-time way to force error handling.
- Callable.call(): declared 'V call() throws Exception' — any checked or unchecked exception thrown inside is caught by FutureTask exactly the same way, stored, and rethrown from get() wrapped in ExecutionException.
- This is precisely why submit() accepts Callable but execute() does not: execute() has no Future to carry a checked exception back to, so the language design forces fire-and-forget tasks to be Runnable-only.

5.2 invokeAll() — Submit a Batch, Wait for All

ExecutorService.invokeAll(Collection<? extends Callable<T>>) submits every task, blocks until ALL of them complete (normally or by throwing), and returns a List<Future<T>> in the same order as the input. Crucially, invokeAll() itself never throws because a task failed — each task's outcome (result or exception) is encapsulated in its own Future, so the caller must iterate the list and call get() on each one to discover which tasks failed.

```
List<Callable<Integer>> tasks = List.of(
    () -> 10 / 2,
    () -> 10 / 0,          // throws ArithmeticException
    () -> 10 / 5
);
List<Future<Integer>> results = pool.invokeAll(tasks);
for (Future<Integer> f : results) {
    try {
        System.out.println("Result: " + f.get());
    } catch (ExecutionException e) {
        System.out.println("Task failed: " + e.getCause());
    }
}
```

5.3 invokeAny() — Return the First Successful Result

invokeAny() submits a batch of Callables and returns the result of whichever one completes successfully first, cancelling the rest. Unlike invokeAll(), a failure here behaves very differently: if a task throws, invokeAny() simply moves on and waits on the others rather than surfacing that exception through a Future. Only if EVERY task in the batch fails does invokeAny() itself throw an ExecutionException directly to the caller — there is no Future involved at all, so there is nothing to call get() on.

6. RejectedExecutionException, Shutdown Behaviour, and CompletableFuture

Exceptions thrown inside a task are only one category of failure a thread pool can produce. Two other situations are commonly confused with in-task exceptions but are fundamentally different: submission-time rejection, and failures surfaced through the newer CompletableFuture API.

6.1 RejectedExecutionException — Failure Before the Task Ever Runs

If a pool's work queue is full and its saturation policy is the default `AbortPolicy`, or if the pool has already been shut down, `execute()/submit()` throw `RejectedExecutionException` synchronously, on the calling thread, before the task is ever queued. This is unrelated to the `run()`-time exception handling discussed above: no worker thread is involved, and the task's code never executes at all.

```
ExecutorService pool = Executors.newFixedThreadPool(2);
pool.shutdown();
try {
    pool.execute(() -> System.out.println("never runs"));
} catch (RejectedExecutionException e) {
    System.out.println("Rejected: pool already shut down");
}
```

6.2 shutdown() vs shutdownNow() and In-Flight Exceptions

- `shutdown()`: stops accepting new tasks but lets already-queued and in-progress tasks finish normally, including their normal exception handling via `afterExecute()/Future`.
- `shutdownNow()`: additionally attempts to interrupt actively running worker threads and drains unstarted tasks back to the caller as a `List<Runnable>`; a task caught mid-execution may see `InterruptedException` if it is blocking, which then flows through the same `execute()/submit()` paths described in Sections 1 and 3.

6.3 CompletableFuture — A More Composable Alternative

`CompletableFuture<T>` (typically produced via `ExecutorService.submit()`-like helpers such as `CompletableFuture.supplyAsync(Supplier, Executor)`) captures a thrown exception the same way `FutureTask` does, but instead of forcing a blocking `get()`, it lets the caller register non-blocking handlers.

```
CompletableFuture<Integer> cf = CompletableFuture.supplyAsync(() -> {
    if (true) throw new RuntimeException("boom-async");
    return 42;
}, pool);

cf.exceptionally(ex -> {
    System.out.println("Handled: " + ex.getMessage());
    return -1; // fallback value
}).thenAccept(value -> System.out.println("Final value: " + value));
```

`exceptionally()` (or the more general `handle()/whenComplete()`) plays the same structural role as catching `ExecutionException` around `future.get()` in Section 3, but does so asynchronously — the calling thread never blocks, and the exception is consumed as part of the reactive pipeline rather than at an explicit `get()` call.

6.4 Consolidated Failure-Path Comparison

Failure Path	When It Happens	How Caller Learns About It
execute()	Exception thrown inside <code>run()</code> while task executes	Only via uncaught-exception handler / console; not returned to caller
submit() + Future	Exception thrown inside <code>call()/run()</code> while task executes	<code>future.get()</code> throws <code>ExecutionException</code>
invokeAll()	One or more tasks throw during the batch	Per-task: iterate returned <code>Futures</code> and call <code>get()</code> on each

Failure Path	When It Happens	How Caller Learns About It
invokeAny()	Some (not all) tasks throw	Ignored internally; only thrown to caller if ALL tasks fail
RejectedExecutionException	Queue full or pool shut down, before task runs	Thrown synchronously on the calling thread at submission time
CompletableFuture	Exception thrown inside <code>supplyAsync()/runAsync()</code> body	<code>exceptionally()/handle()/whenComplete()</code> callbacks, non-blocking